UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Lahore, Pakistan

Department of Computer Engineering

PROJECT REPORT

Inventory Management System

A Desktop Application Built with Python & PyQt5

Project Name	Inventory Management System
Submitted By	Mishkat e Noor
Roll No	2026-CYS(S)-06
Course	Programming Fundamentals Lab
Session	28-06-2026
Submitted To	Hafiz Mubashir Imran

1. Project Name & Overview

Project Name	Inventory Management System
Technology Stack	Python 3.x, PyQt5 5.15.11, Qt Designer, JSON
Application Type	Desktop GUI Application
Data Storage	Local JSON file (data/inventory.json)
Platform	Windows / Linux / macOS (Cross-platform)

The Inventory Management System is a desktop application developed using Python and the PyQt5 graphical framework. It provides a structured, user-friendly interface for managing product inventory — enabling users to add, view, search, update, and delete items. All data is stored persistently in a local JSON file, ensuring continuity between sessions without requiring a database server.

The application demonstrates practical application of GUI programming, event-driven design, and file-based data persistence — core concepts covered in the Programming Fundamentals Lab course.

2. Problem Statement

Many small businesses, warehouses, and academic labs continue to manage their inventory through manual methods — paper registers, Excel spreadsheets, or informal logs. While these approaches may work at a small scale, they introduce significant operational risk and inefficiency as inventory grows.

Core Challenges Identified:

- Inaccurate stock records caused by manual data entry errors
- Overstocking or understocking due to lack of real-time visibility into inventory levels
- Time-consuming search and retrieval of product information from large registers or sheets
- No automatic saving or audit trail, leading to data loss and inconsistency
- Difficulty in updating or correcting records without risking accidental overwrites
- Inaccessibility for non-technical users who cannot work with raw spreadsheets or databases

These challenges collectively reduce efficiency, increase operational costs, and create opportunities for errors that directly impact financial performance. There is a clear need for a simple, automated, and reliable desktop tool that non-technical users can operate with minimal training.

3. Objectives

This project was developed with the following clearly defined objectives:

Primary Objectives:

1. Develop an automated desktop-based inventory tracking application that replaces manual record-keeping.
2. Maintain accurate and real-time stock records with automatic persistent storage after every operation.
3. Reduce human errors through structured data entry forms, input validation, and confirmation dialogs.
4. Provide fast, real-time search and retrieval of product information by name.
5. Enable full CRUD operations (Create, Read, Update, Delete) on inventory items through an intuitive interface.
6. Build an application accessible to users without technical backgrounds using a graphical user interface.

Secondary Objectives:

- Apply and demonstrate Python programming concepts taught in the Programming Fundamentals Lab.
- Practice modular software design by separating GUI logic from data management logic.
- Gain hands-on experience with Qt Designer and PyQt5 signal-slot architecture.
- Produce clean, maintainable, and well-structured source code.

4. Working Methodology

The project was developed using a structured, phase-based approach. Each phase was completed sequentially to ensure a stable and well-tested final product.

Phase	Stage	Activities Performed
1	Requirement Analysis	Identified core features needed: CRUD operations, search, persistent storage, and a graphical interface. Defined user stories and system scope.
2	System Design	Designed the three-layer architecture (entry point, GUI layer, data layer). Created the UI layout in Qt Designer using drag-and-drop widget placement.

Phase	Stage	Activities Performed
3	Development	Implemented inventory_data.py for JSON I/O, inventory_gui.py for all GUI logic and event handling, and main.py as the application entry point.
4	Testing	Manually tested all CRUD operations, edge cases (empty name, no selection, confirm delete), and data persistence across application restarts.
5	Documentation	Wrote README.md, inline code comments, and this project report summarising the complete development process.

4.1 Architecture & Data Flow

The application follows a clean separation of concerns across three modules:

- main.py initialises the QApplication, creates the InventoryWindow, and launches the event loop.
- inventory_gui.py loads the .ui layout at runtime, connects button signals to handler slots, and manages all user interaction, table rendering, and form state.
- inventory_data.py handles all file I/O — loading the JSON on startup, saving after every mutation, and generating unique sequential item IDs.

4.2 Technology Justification

Technology	Reason for Selection
Python 3.x	Readable syntax ideal for beginner-level projects; rich standard library; cross-platform support.
PyQt5	Mature, widely-used GUI framework with Qt Designer integration; supports all major desktop platforms.
Qt Designer	Visual layout editor eliminates manual widget positioning code; produces .ui files loadable at runtime.
JSON	Lightweight, human-readable format; no external database required; native Python support via the json module.

5. Results

Upon completion, the application was tested across all defined use cases. The following outcomes were confirmed:

Test Case	Status	Notes
Add a new item with name, quantity, and price	PASS	Item appears in table immediately with auto-assigned ID.
Attempt to add item with empty name	PASS	Warning dialog shown; item not saved.
Select item from table and update its details	PASS	Form populates correctly; update saves and refreshes table.
Delete an item with confirmation	PASS	Confirmation dialog shown; item removed on Yes.
Cancel delete from confirmation dialog	PASS	Item remains in table; no data changed.
Search by product name (partial match)	PASS	Table filters in real time; case-insensitive.
Restart application and verify data persists	PASS	All items reload from inventory.json correctly.
Click Update without selecting a row	PASS	Info dialog shown; no crash or error.

All eight test cases passed successfully. The application operates stably without crashes or data loss across normal usage scenarios. The status bar provides real-time feedback after every operation, improving user confidence and transparency.

6. Conclusion

The Inventory Management System project successfully achieved all primary and secondary objectives. A fully functional, cross-platform desktop application was built using Python and PyQt5 that allows users to manage inventory items through an intuitive graphical interface.

The project demonstrated practical application of key programming concepts including object-oriented design, event-driven programming, file I/O, data validation, and modular code architecture. The clean separation between the GUI layer and the data layer makes the codebase straightforward to maintain and extend.

Potential Future Enhancements:

- Barcode scanning integration for rapid item lookup and stock updates
- Category and supplier fields with filter and sort capabilities
- Low-stock alert system with configurable threshold notifications

- PDF and CSV report generation for management review
- Cloud or database backend (SQLite or Firebase) for multi-user support
- Analytics dashboard showing total inventory value, top items, and stock trends

7. References

- Python Software Foundation. Python 3 Official Documentation. https://docs.python.org/3/
- The Qt Company. PyQt5 Reference Guide. https://doc.qt.io/qtforpython-5/
- Riverbank Computing. PyQt5 Documentation. https://www.riverbankcomputing.com/software/pyqt/
- The Qt Company. Qt Designer Manual. https://doc.qt.io/qt-5/qtdesigner-manual.html
- Python json module. Python Standard Library. https://docs.python.org/3/library/json.html
- University of Engineering and Technology, Lahore. Course: Programming Fundamentals Lab, 2026.